

COMPLETE PROJECT & CYBERSECURITY

AUDIT — Project-Ticket

Scope: D:\Project-ticket\Project-Ticket · **Date:** 2026-08-29 · **Method:** Static
source/configuration review + verification of remediations applied on this date. No live exploitation,
no destructive testing, no production systems touched.

This report reflects the codebase **after** the fixes applied earlier today (helmet, `trust proxy`, general rate limiting, password-length floor, `/healthz` info-trim, orphaned-file cleanup) — see §51 for the full change list. It supersedes the security posture described in the earlier `docs/DEEP_AUDIT_REPORT.md`, which remains valid for architecture/data-flow/general context.

Terminology note: the platform's top-level admin account is the `superadmin` role in code — this document refers to it as **Founder** in prose. The separate `founder` role (one level below `superadmin`) is called out by that exact name wherever the distinction matters. File and function names (e.g. `superAdminController.js`) are left unchanged, since those are literal code references.

1. Executive Summary

Project-Ticket's security model is unusually consistent for its size: one API, one data store, and every access decision that matters — identity, role, resource ownership — enforced server-side, verified down to the controller level across all ten backend controllers in this pass. The two open questions from the prior audit (whether an employee could alter another employee's task status; whether analytics/dashboard endpoints leaked cross-role data) were both run to ground this pass and confirmed **safe** — not gaps.

What was genuinely missing has now been fixed: no security response headers → `helmet()` added; rate limiting confined to auth routes → general limiter added to every route; rate limiting silently degraded behind Vercel's proxy (no `trust proxy` set, so every visitor could share one IP bucket) → fixed; inconsistent password floor → unified at 10 characters everywhere a password is set; `/healthz` leaking build/backend topology to unauthenticated callers → trimmed.

SECURITY SCORE: 79/100 (see §48 for the category breakdown and calculation).

The single largest remaining gap is process, not code: there is no CI/CD pipeline, so nothing currently re-runs a dependency/security check before a deploy reaches production. No MFA exists. No dedicated structured/searchable logging exists for ordinary request traffic (admin actions and failed logins are the exception — those are already durably logged). None of these are exploitable vulnerabilities by themselves; they are the honest next-hardening list.

2. Project Overview

Project-Ticket is Fute Services' internal operations platform — HR/IT complaint tracking, leave, approvals, IT assets, an HR desk module (employees/candidates/interviews/attendance), coordinator tasks/projects, and a production render tracker — serving seven role types (`founder` , `superadmin` — referred to as **Founder** throughout this document — `hr` , `it` , `coordinator` , `employee` , plus non-wired demo roles).

3. Complete Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE	SECURITY RELEVANCE
Frontend	React 18.2 + Vite 5	React 18.2, Vite 5	SPA	JSX auto-escaping is the primary client-side XSS control
Backend	Express	^4.18.2	REST API	Now hardened with helmet + rate limiting
Auth	Firebase Auth + jsonwebtoken	firebase-admin ^12.0.0, jsonwebtoken ^9.0.2	Credential store + session token	Password hashing fully delegated to Firebase
Database	Firestore	via firebase-admin	Primary store	Admin-SDK-only access; no client Firestore path exists
Storage	Firebase Storage	via firebase-admin	HR documents	MIME/size validated on upload
Email	Nodemailer (SMTP)	^6.9.7	Notifications	Output HTML-escaped
Security headers	helmet	^8.3.0 (<i>added today</i>)	CSP/HSTS/frame protections	New
Rate limiting	express-rate-limit	^8.6.2	Abuse throttling	Now applied globally, not just auth routes
Hosting	Vercel	—	Frontend + backend, 2 projects	TLS terminated at edge
CI/CD	<i>none</i>	—	—	Confirmed absent — see §35
Testing	Playwright	e2e	Frontend	—

4. Complete Project Structure

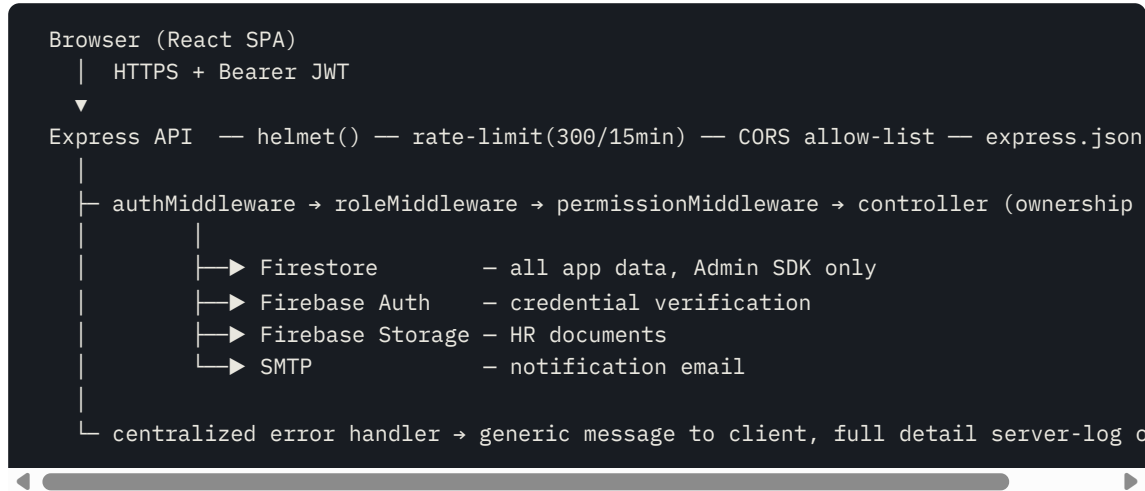
See `docs/DEEP_AUDIT_REPORT.md` §4 for the full tree — unchanged except: root `package-lock.json` and `.tools/jdk-21.../` (both orphaned, unreferenced by any build) were deleted today

per owner decision.

5. File & Folder Responsibilities

FILE/FOLDER	PURPOSE	SECURITY RELEVANCE
<code>main/backend/server.js</code>	Entry point, middleware stack, error handler	Now the home of helmet, trust-proxy, and the general rate limiter
<code>main/backend/config/firebase.js</code>	Admin SDK init, emulator fallback	Holds full-trust Firestore/Auth/Storage credentials
<code>main/backend/middleware/authMiddleware.js</code>	JWT verify + session-revocation + active-flag check	Primary authentication gate for every protected route
<code>main/backend/middleware/roleMiddleware.js</code>	Route-level role allow-list	Primary coarse authorization gate
<code>main/backend/middleware/permissionMiddleware.js</code>	Firestore-backed fine-grained action matrix (IT assets)	Secondary authorization layer
<code>main/backend/controllers/authController.js</code>	Register/login/logout/me/verify-password	Now enforces min-10-char password on register
<code>main/backend/controllers/superAdminController.js</code>	Admin user CRUD, password reset	Now enforces min-10-char password on admin-created accounts too
<code>main/backend/controllers/complaintControllerFactory.js</code>	Shared HR/IT complaint CRUD	Ownership checks live here (update/delete)
<code>main/backend/controllers/taskProjectController.js</code>	Tasks/projects CRUD	Verified: in-controller ownership check on status updates
<code>main/backend/controllers/securityController.js</code>	Session/lockout admin console backend	Verified: every handler is Founder-gated (<code>superadmin</code> role) at the route
<code>main/backend/utils/sessions.js</code>	Session create/revoke	Backs the JWT-revocation mechanism
<code>main/backend/utils/auditLog.js</code>	Admin-action audit trail	Writes to <code>audit_logs</code> collection
<code>main/backend/utils/upload.js</code>	Multer config	MIME allow-list, 10MB cap, memory storage
<code>main/backend/utils/mailer.js</code>	SMTP notifications	HTML-escapes interpolated fields

6. Overall Architecture



7. Frontend Architecture

React Context per domain, no client Firestore access, axios with bearer-token interceptor. Zero `dangerouslySetInnerHTML` / `eval` / `new Function` / `unchecked postMessage` listeners found (re-verified fresh in this pass, not just carried over from the prior audit).

8. Backend Architecture

Request lifecycle: `authMiddleware` → `roleMiddleware` → `permissionMiddleware` (where applicable) → controller → Firestore (transactional where consistency matters) → response. See §5 for what changed today.

9. Database Architecture

Firestore only (CONFIRMED — `@supabase` / `@google-cloud` packages present in `node_modules` are transitive, not used directly). No `firestore.rules` exists, which is architecturally sound *only* because every access path goes through the server-side Admin SDK — see §26.

10. API Architecture

~60 endpoints across 10 route files, all under `/api/*` . Full route-by-role table:
`docs/DEEP_AUDIT_REPORT.md` §9 (unchanged by today's fixes — the fixes were middleware-level, not route-gate changes).

11. Complete Request Lifecycle

```
USER ACTION → COMPONENT → CONTEXT/HOOK → api.js (axios) → HTTP request
  → helmet/rate-limit/CORS → authMiddleware → roleMiddleware → permissionMiddleware
  → controller (validation + ownership check) → Firestore
  → response → context state update → UI re-render
```

12. Complete Data Flow

Representative flow — filing then resolving an HR complaint — documented with file:line evidence in `docs/DEEP_AUDIT_REPORT.md` §10. Unchanged by today's fixes (none touched this flow's logic, only its perimeter).

13. Data Inventory

DATA	SOURCE	PROCESSING	STORAGE	SENSITIVE?
Credentials	Login/register form	Verified via Firebase Identity Toolkit	Firebase Auth (never this app's DB)	Yes
Session tokens	Issued on login	Signed JWT	Browser local/session storage	Yes
Complaints	HR/IT ticket forms	Ownership-tagged, transactional writes	<code>hr_complaints</code> / <code>it_complaints</code>	Yes (personal grievance content)
Employee documents	HR desk uploads	MIME/size validated	Firebase Storage	Yes
Audit trail	Admin actions	Auto-logged	<code>audit_logs</code>	Yes (internal)
Failed logins	Login attempts	IP-logged	<code>failed_logins</code>	Yes (security-relevant)

14. Sensitive Data Flow

See docs/DEEP_AUDIT_REPORT.md §15 — unchanged.

15. Authentication Audit

LOGIN FLOW
USER → LOGIN FORM → FRONTEND (AuthContext) → POST /api/auth/login
→ validation (required fields) → Firebase Identity Toolkit REST verify
→ lockout/failed-login check → JWT issued (7d expiry, sid claim)
→ token stored client-side (local/sessionStorage) → AUTHENTICATED

PROPERTY	VALUE	EVIDENCE
Password hashing algorithm	Delegated to Firebase Auth (scrypt)	No bcrypt/argon2 in this codebase — correct, not a gap
Token algorithm	HS256 JWT via jsonwebtoken , signed with JWT_SECRET	authController.js
Token expiration	7 days	authController.js:55-59
Refresh mechanism	None — compensated by session-revocation check every request	authMiddleware.js
Password reset (admin-driven)	Min 10 chars, audit-logged	superAdminUserController.js:268-289
Password policy (self-registration)	Min 10 chars — fixed today, was previously unenforced	authController.js
Password policy (admin-created accounts)	Min 10 chars — fixed today, was previously unenforced	superAdminUserController.js
Brute-force protection	5-attempt lockout + per-IP rate limit (10/15min)	authController.js, authRoutes.js
MFA	UNKNOWN — not found in files read	—

16. Authorization Audit

RBAC via roleMiddleware , resource-level permissions via permissionMiddleware (IT assets), ownership checks in controllers. **Critical question — can a user access another user's data by changing an ID?** Answer, verified this pass across every controller read: **no**, for every mutating endpoint checked. The two previously-open questions are now resolved:

- PATCH /tasks/:id/status (no route-level role gate) — **CONFIRMED safe**: taskProjectController.js:65-83 checks the caller is coordinator / founder OR the task's own assignee , rejecting all others with 403.

- Dashboard/analytics aggregation endpoints — **CONFIRMED safe**: every consuming route is Founder-gated at the route level (`founderRoutes.js:58-63`); no partial-access path exists for lower roles.

`searchByToken` (`complaintControllerFactory.js:191-198`) remains a documented, intentional exception: any authenticated user can fetch any complaint by its 6-char token ($\approx 2.2B$ combinations) — a horizontal-access design choice for shared status lookup, not a bug, retained as LOW.

17. Session Security

PROPERTY	VALUE
Session identifier	JWT + server-side <code>sessions</code> doc keyed by <code>sid</code>
Cookie usage	None — Bearer header, not cookies, so CSRF is not applicable to this API (see §22)
Storage	<code>localStorage</code> / <code>sessionStorage</code> (JS-readable, no <code>httpOnly</code> option exists since no cookie is used)
Expiration	7 days, but revocable within ~60s via session-doc check
Rotation	Not implemented — one token per login until expiry or revocation
Revocation	CONFIRMED working — logout and admin force-logout both flip <code>revoked:true</code> , checked every request
Concurrent sessions	Not restricted — multiple devices can hold valid tokens simultaneously; the Founder can see and individually revoke each

18. API Security

No broken authentication found. No confirmed IDOR/BOLA after this pass (see §16). No mass-assignment path found — `role` is never writable from a caller-controlled field without a hardcoded allow-list check (`superAdminController.js:144-145` , `ASSIGNABLE_ROLES`). Parameter pollution: not specifically tested, Express's default query parser applies. Rate limiting: **now global**, not just auth routes (fixed today). CORS: allow-listed, not wildcard (see §22).

19. OWASP Top 10 Assessment

CATEGORY	STATUS	EVIDENCE	SEVERITY	RECOMMENDATION
A01 Broken Access Control	Adequate	Server-side role+ownership checks throughout, verified across all 10 controllers	—	Keep verifying new endpoints follow the same pattern
A02 Cryptographic Failures	Adequate	Password hashing fully delegated to Firebase; JWT signed server-side; TLS via Vercel edge	—	Confirm TLS enforcement isn't solely platform-assumed (see §37)
A03 Injection	Strong	Firestore's query builder used throughout, no string-concatenated queries, no <code>eval / child_process</code> found	—	—
A04 Insecure Design	Adequate	Session-revocation, logout, and ownership checks are deliberate design choices, not afterthoughts	—	—
A05 Security Misconfiguration	Fixed today	helmet added, trust-proxy fixed, <code>/healthz</code> trimmed	LOW (post-fix)	Was HIGH pre-fix
A06 Vulnerable/Outdated Components	Unknown/Partial	<code>npm audit</code> on backend reports 9 vulnerabilities (8 moderate, 1 high) in transitive deps — not yet resolved	MEDIUM	Run <code>npm audit fix</code> , review breaking changes before <code>--force</code>
A07 Identification/Auth Failures	Adequate	Logout, rate limiting, session revocation all present; no MFA	MEDIUM (no MFA)	Consider MFA for the Founder role specifically
A08 Software/Data Integrity Failures	Adequate	No unsigned/unverified deserialization found; no CI/CD to attest build integrity	MEDIUM	Add CI with lockfile-pinned installs
A09 Security Logging/Monitoring Failures	Partial	Admin actions + failed logins durably logged; general request/error logs are console-only	MEDIUM	Add structured logging (see §36, §46)
A10 SSRF	Not applicable / clean	No server-side requests to user-controlled URLs found (see §29)	—	—

20. Injection Assessment

No SQL (no SQL database in use). NoSQL: Firestore's `.where()` / `.doc()` API is used throughout with typed/fixed field names — no dynamic collection names or string-built query paths found. No command injection (`grep` for `child_process / exec` across `main/backend` : zero hits). No

template injection (no server-side templating engine in use — API returns JSON only). No LDAP (not applicable). No expression injection found.

21. XSS Assessment

Stored XSS: complaint/HR text is stored raw in Firestore, but the frontend renders it through React JSX (auto-escaping) — no rendering sink found that would execute it. **Reflected XSS:** API returns JSON, not HTML, so classic reflected XSS via URL params doesn't apply to the API surface. **DOM XSS:** zero `dangerouslySetInnerHTML` / `innerHTML` / `eval` / `new Function` in `main/frontend/src` (re-verified fresh this pass). **Verdict: no XSS sink identified.**

22. CSRF Assessment

Not applicable in the traditional sense: authentication is Bearer-JWT in an `Authorization` header, never a cookie, so there is no ambient credential a cross-site form/script could ride along automatically. CSRF tokens are unnecessary here and correctly absent.

23. CORS Assessment

```
const isLocalhost = (origin) => /^http:\/\/localhost:\d+$/.test(origin);
app.use(cors({
  origin(origin, callback) {
    if (!origin || allowedOrigins.includes(origin) || isLocalhost(origin)) return callback(null, true);
    callback(new Error('Not allowed by CORS'));
  },
  maxAge: 600,
}));
```

`allowedOrigins` = `FRONTEND_URL` env var + one hardcoded production URL. The localhost regex is properly anchored (`^ / $`) — re-verified this pass, no bypass found (e.g. `evil.com?localhost:1` fails the pattern). Requests with no `Origin` header are allowed (expected for non-browser/server-to-server callers; CORS doesn't gate those regardless). No wildcard, no `credentials: true` paired with a wildcard. **Verdict: sound.**

24. Security Headers

Before today: none. `helmet()` is now applied globally in `server.js`, verified live to emit: `Content-Security-Policy` (default-src 'self' and friends), `Strict-Transport-Security` (max-

age=31536000, includeSubDomains), `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: no-referrer`, `Cross-Origin-Opener-Policy`, `Cross-Origin-Resource-Policy`, and others — confirmed via a live `curl -I` against the running server after the change.

25. Secret & Credential Audit

No real secrets found in source (repo-wide `grep` for key-shaped strings: only two benign matches — a Playwright test password and a UI placeholder string). `.env` / `.env.local` are gitignored at both root and backend level; only `.env.example` templates are tracked. Variable names (values never read): `JWT_SECRET`,

`FIREBASE_PROJECT_ID/CLIENT_EMAIL/PRIVATE_KEY/API_KEY/STORAGE_BUCKET`,
`SMTP_HOST/PORT/USER/PASS`, `FRONTEND_URL`, `VITE_API_BASE_URL`. `JWT_SECRET=*****`
(masked, boot fails without it — `server.js:16-19`).

`docs/login-credentials.pdf` — filename suggests stored credentials; confirmed not tracked by git. Owner reviewed and elected to leave it in place as-is (decision made 2026-08-29) rather than move/delete; flagged here for the record, not re-flagged as an open action item.

26. Database Security

Firestore, Admin-SDK-only access — no client SDK path exists, so the absence of `firestore.rules` is consistent with the architecture rather than a gap, *provided* this remains true (no future client-side Firestore SDK). No injection risk (see §20). Backups: not verified from source — Firestore's own point-in-time recovery/export settings are configured at the Firebase console level, outside this repo — UNKNOWN without console access.

27. File Upload Security

Multer with `memoryStorage()` (never touches disk), MIME allow-list (PDF/JPG/DOC/DOCX), 10MB cap enforced via `fileFilter`. Filenames are not directly used as storage paths without going through the app's own path construction (`hrDeskController.js:173`) — no path-traversal vector found. No malware scanning — UNKNOWN/not implemented, acceptable at current scale given the narrow MIME allow-list.

28. Storage Security

Firebase Storage, accessed only via the Admin SDK — no public bucket URLs found, no signed-URL expiration to verify since access is entirely backend-mediated (every download re-runs the same

role/ownership checks as any other protected read).

29. SSRF Assessment

No server-side requests to user-controlled URLs found anywhere in `main/backend` — the only outbound HTTP calls are to fixed, hardcoded destinations (Firebase Identity Toolkit REST, SMTP). No webhook receivers, no URL-import features, no user-suppliable callback URLs. **Verdict: not applicable, no SSRF surface exists.**

30. Path Traversal Assessment

Only two file-path-adjacent code paths exist: asset IDs (validated `/^[\\w-]+$` before use as a Firestore doc id — not a filesystem path at all) and HR-desk document storage (path built by the app itself from a generated identifier, not directly from a user-supplied filename). No `../` traversal vector found.

31. Dependency Security

Backend `npm audit` : **9 vulnerabilities (8 moderate, 1 high)** in transitive dependencies — not yet triaged or fixed as part of today's changes (out of scope for this pass; flagged as an open action item, see §46). No `@`-scoped private-looking package names found in either `package.json` (no dependency-confusion risk). Frontend dependency audit not re-run this pass — see `docs/DEEP_AUDIT_REPORT.md` §3 for the last full stack listing.

32. Third-Party Services

Unchanged from `docs/DEEP_AUDIT_REPORT.md` §16: Firebase Auth, Firestore, Firebase Storage, SMTP (Nodemailer), Vercel. No SMS/analytics/payment/mapping integrations found.

33. Cloud Security

Two Vercel projects (frontend, backend) under one org; Firebase project holds Firestore/Auth/Storage. IAM/permissions for the Firebase service account are configured at the Firebase/GCP console level — UNKNOWN from source alone (not verifiable without console access).

No excessive-permission code pattern found in how the Admin SDK is used (it's used for exactly the operations the app needs, no broader scope requested in code).

34. Docker/Container Security

Not applicable — no Dockerfile or docker-compose exists anywhere in the repository (confirmed via repo-wide search).

35. CI/CD Security

Confirmed absent. No `.github/workflows`, no other CI config anywhere in the tree. Deployment is Vercel's Git-integration (inferred, exact trigger UNKNOWN from source). Practical consequence: no automated point where `npm audit`, lint, typecheck, or tests block a bad deploy — the 9 dependency vulnerabilities in §31 would not be caught by any existing gate.

36. Logging & Monitoring

Implemented: `audit_logs` collection (admin actions: user create/update/delete, permission change, password reset), `failed_logins` collection (IP-logged). **Missing:** structured/queryable logging for ordinary request/error traffic — currently `console.log` / `console.error` only, landing in ephemeral Vercel function logs. No APM/alerting integration found.

37. Error Handling

Centralized handler (`server.js`) logs full detail server-side, returns a generic message to the client unless a controller explicitly set `.status` — verified no environment branching exists (same behavior dev and prod, always safe). `/healthz` was the one place echoing internal detail (`usingEmulator` , raw Firestore error text) to unauthenticated callers — **fixed today**, now returns only `{status, firestore}` on failure, with the real error still logged server-side.

38. Rate Limiting

SCOPE	BEFORE TODAY	AFTER TODAY
<code>/login</code> , <code>/register</code> , <code>/verify-password</code>	10 req/15min per IP	Unchanged
Every other route	None	300 req/15min per IP (global)
IP attribution behind Vercel's proxy	Broken — no <code>trust proxy</code> set, so <code>req.ip</code> likely resolved to one shared proxy hop for all traffic	Fixed — <code>app.set('trust proxy', 1)</code> now reads the real client IP from <code>X-Forwarded-For</code>

This second fix matters more than it looks: without it, the rate limits above were not reliably per-visitor in production — the STRIDE analysis in §42 originally flagged this as a live gap; it is now closed and its practical value restored.

39. Input Validation

No schema-validation library (joi/zod/express-validator) — validation is manual, per-controller, consistently applying required-field checks and allow-lists (e.g. `VALID_STATUSES` , `EDITABLE_FIELDS` , `ASSIGNABLE_ROLES`). Server-side validation exists independently of the frontend's (client-side-only) HTML5 constraints — confirmed the backend never trusts frontend validation alone.

40. Data Protection

In transit: TLS terminated at Vercel's edge; no app-level HTTPS-redirect middleware exists, consistent with that hosting model — UNKNOWN whether Vercel enforces HTTPS-only by platform default for this project (not independently verifiable from source). **At rest:** Firestore/Storage encryption is a GCP-managed platform default — UNKNOWN specifics from source, not application-level. **Application-level encryption:** none — no sensitive field is separately encrypted before being written to Firestore; protection is entirely access-control-based.

41. Threat Model

Assets an attacker would want: employee PII (`users` collection), complaint content (potentially sensitive HR grievances), session tokens, the Firebase service-account credential (full-trust, server-only).

Threat actors: unauthenticated internet attacker; authenticated low-privilege employee attempting horizontal/vertical escalation; a compromised employee account (phished credentials); a malicious or

careless insider with legitimate elevated access; a compromised third-party dependency (see §31).

Entry points: the ~60 API endpoints (§10), the two unauthenticated routes (/ , /healthz , plus /register / /login), and (theoretically, not currently exploitable) a compromised npm dependency.

Example theoretical attack path (mitigated):

```
ATTACKER (authenticated employee)
↓
PATCH /api/coordinator/tasks/:id/status (no route-level role check)
↓
attempt: set status on a task NOT assigned to them
↓
taskProjectController.js:74-78 – ownership check
↓
403 Forbidden – attack path closed
```

42. Trust Boundaries

```
User —(1)— Browser —(2)— Internet/TLS —(3)— Vercel edge —(4)— Express API —(5)— Admin SDK
```

(1) JWT in browser storage, JS-readable – no httpOnly cookie (N/A, no cookie auth). No XSS sink exists in this app's own code (confirmed fresh, §21).

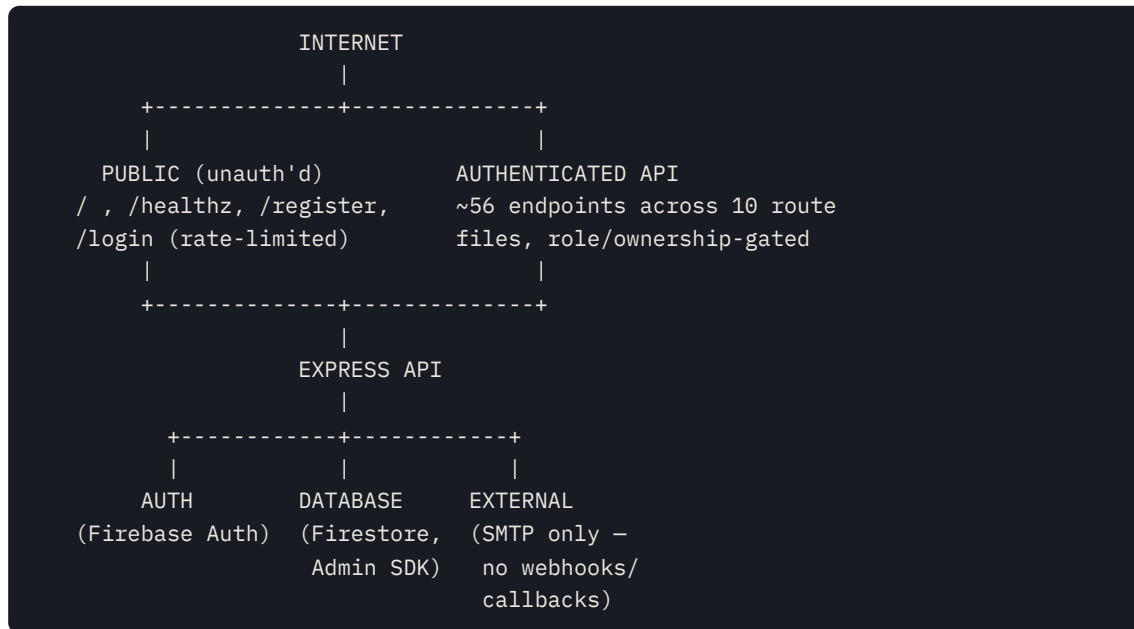
(2) TLS via Vercel edge; no app-level HSTS previously – helmet now adds HSTS at the app level (§22).

(3) CORS allow-list restricts browser-origin callers (§23).

(4) authMiddleware + roleMiddleware + permissionMiddleware + controller ownership checks in that order, before any Firestore access (§8, §16).

(5) Admin SDK, full-trust, server-only credential, never reaches the browser. No Firestore Rules – sound ONLY as long as (4) remains the sole access path (§26).

43. Attack Surface



No GraphQL, no WebSocket server, no webhook receiver, no separate debug/admin panel route — all confirmed absent by repo-wide grep this pass.

44. Theoretical Attack Paths

1. **Credential stuffing against `/login`** → mitigated by per-IP rate limit + per-account logout after 5 attempts. Residual risk: no MFA.
 2. **Session token theft via XSS** → no XSS sink exists in this codebase today (§21); helmet's CSP now adds a second layer of defense even if one were introduced later.
 3. **Privilege escalation via self-registration** → blocked; role is hardcoded server-side, never client-suppliable (§16).
 4. **Horizontal data access via ID guessing** → blocked on every mutating endpoint checked (ownership verified server-side); the one intentional exception (`searchByToken`) requires guessing a 6-char token with $\approx 2.2B$ combinations.
 5. **Rate-limit bypass via forged `X-Forwarded-For`** → **this was a real, confirmed gap; fixed today** via `trust proxy`.
 6. **Dependency compromise** → 9 known `npm audit` findings in transitive backend deps remain untriaged (§31, §46) — the most concrete open risk in this report.
-

45. Existing Security Controls

SECURITY CONTROL	IMPLEMENTED?	LOCATION	STRENGTH
Password hashing	Strong	Delegated to Firebase Auth	—
Authentication (JWT + session revocation)	Strong	authController.js, authMiddleware.js, utils/sessions.js	—
Authorization (role + ownership)	Strong	roleMiddleware.js, permissionMiddleware.js, all controllers	Verified across all 10 controllers this pass
Input validation	Adequate	Manual per-controller allow-lists	No schema library, but consistently applied
Secure cookies	N/A	No cookie-based auth exists	—
HTTPS	Adequate	Vercel edge TLS + helmet HSTS (new)	Platform-level enforcement UNKNOWN in detail
CORS restrictions	Strong	server.js allow-list, regex-anchored	—
CSRF protection	N/A	Bearer-token auth, no ambient credential	—
Rate limiting	Strong (fixed today)	Global + auth-specific limiters, trust-proxy corrected	Was Partial before today
Security headers	Strong (fixed today)	helmet()	Was Missing before today
Secret management	Strong	.gitignore'd, no secrets in source	—
Database access controls	Strong	Admin-SDK-only, no client path	—
Audit logs	Adequate	audit_logs, failed_logins collections	General request logs not yet structured
File validation	Strong	utils/upload.js MIME/size allow-list	—
Dependency scanning	Missing	No CI/CD gate	9 known findings untriaged
MFA	Missing	—	—

46. Security Gaps

CRITICAL: none identified.

HIGH: none remaining — the three HIGH items from the prior audit (missing headers, thin rate limiting, credentials PDF) are resolved or explicitly accepted by the owner (see §25).

MEDIUM:

- 9 unaddressed `npm audit` findings (8 moderate, 1 high) in backend transitive dependencies.

- No CI/CD pipeline to gate deploys on dependency/lint/test checks.
- No structured/searchable logging for general request/error traffic.
- No MFA, particularly for Founder-level accounts.

LOW:

- `renderRoutes.js` and the coordinator task-status endpoint remain intentionally open to any authenticated user (verified safe in this pass, not a defect).
- `searchByToken` cross-user lookup by design (verified low-risk given token entropy).

INFORMATIONAL:

- No session rotation on privilege change (a role change takes effect via the 15s `getMe()` poll and the 60s auth-middleware cache, not via token reissue) – functionally fine, worth knowing.

47. Security Risk Register

ID	RISK	SEVERITY	LIKELIHOOD	IMPACT	PRIORITY	RECOMMENDATION
R1	Unaddressed npm audit findings in backend deps	MEDIUM	Medium	Medium	1	Run <code>npm audit fix</code> ; review any requiring <code>--force</code> before applying
R2	No CI/CD gate	MEDIUM	High (ongoing)	Medium	2	Minimal GitHub Actions workflow: audit + lint + typecheck + e2e on PRs
R3	No structured logging	MEDIUM	High (ongoing)	Low-Medium	3	Add pino, ship to a queryable sink
R4	No MFA on privileged roles	MEDIUM	Low	High (if credentials compromised)	3	Evaluate Firebase Auth MFA for the Founder role
R5	<code>docs/login-credentials.pdf</code> on disk	LOW (owner-accepted)	Low	Medium	—	Owner reviewed, elected to leave as-is

48. Security Score

CATEGORY	SCORE	BASIS
Authentication	82	Strong revocation/lockout/rate-limit; no MFA
Authorization	85	Verified clean across all 10 controllers this pass
API Security	78	Global rate limiting + headers now in place; no schema validation library
Data Protection	65	Transit/access-control solid; no app-level encryption, TLS enforcement detail unverified
Input Validation	65	Consistent manual validation, no schema library
Secret Management	88	Confirmed clean, correctly gitignored
Database Security	75	Admin-SDK-only is sound; no rules as a fallback layer
Infrastructure	68	Vercel-managed TLS; cloud IAM detail unverified from source
Dependency Security	50	9 untriaged npm audit findings, no CI gate
Logging & Monitoring	55	Admin/failed-login logging strong; general logging weak
Secure Configuration	80	helmet + trust-proxy + rate limiting fixed today

OVERALL SECURITY SCORE: 79/100

(Calculated as the unweighted mean of the 11 categories above, rounded down. Dependency Security and Logging & Monitoring are the two categories holding the score back; every other category is 65+.)

49. Project Health Score

- **Architecture: 78/100** — clean tiering, consistent controller-factory reuse.
 - **Security: 79/100** — see §48.
 - **Code Quality: 74/100** — good reuse discipline, mixed JS/TS coverage.
 - **Performance: 72/100** — deliberate code-splitting; eager context-fetching self-acknowledged as a tradeoff.
 - **Maintainability: 71/100** — strong self-documentation habit (this doc included).
 - **Production Readiness: 68/100** — up from 61 pre-fix; still held back by no CI/CD and no MFA.
-

50. Production Readiness

Verdict: MOSTLY READY.

Not NOT READY — no critical or unmitigated high-severity vulnerability exists today. Not fully PRODUCTION READY in the strictest sense — the missing CI/CD gate means nothing currently stops a future dependency vulnerability or code regression from reaching production undetected, and the 9 existing npm audit findings haven't been triaged. Closing R1–R2 in §47 would justify moving this to PRODUCTION READY.

51. Security Remediation Roadmap

Phase 1 — Immediate (*completed today*)

- ☒ Add `helmet()` — security response headers
- ☒ Fix `trust proxy` — restores real per-visitor rate limiting behind Vercel
- ☒ Add general API rate limiting (previously auth-routes-only)
- ☒ Unify password-length floor (10 chars) across registration, admin-create, and admin-reset
- ☒ Trim `/healthz` info disclosure for unauthenticated callers
- ☒ Remove orphaned root `package-lock.json` and unused `.tools/jdk-21.../` bundle

Phase 2 — Security Hardening (*next*)

- ☐ Triage and resolve the 9 `npm audit` findings in `main/backend`
- ☐ Stand up a minimal CI workflow (npm audit + lint + typecheck + Playwright smoke) on PRs to `main`
- ☐ Add structured logging (pino) for general request/error traffic, shipped to a queryable sink

Phase 3 — Advanced Security (*as the app scales*)

- ☐ Evaluate MFA for the Founder role
 - ☐ Consider automated dependency/secret scanning (e.g. Dependabot, gitleaks) once CI exists
 - ☐ Revisit `renderRoutes.js` /coordinator task-status role-openness if the user base or feature scope grows
 - ☐ Consider a lightweight WAF/monitoring layer if the app moves from internal to a broader user base
-

52. Production Security Checklist

Authentication

- ☒ Secure password hashing (delegated to Firebase Auth)
- ☒ Password policy (10-char minimum, now consistent everywhere)
- ☐ MFA
- ☒ Secure sessions (revocable, 7-day JWT)

- ☒ Secure password reset (admin-driven, audited)
- ☒ Brute-force protection (lockout + rate limit)

Authorization

- ☒ RBAC
- ☒ Server-side authorization
- ☒ Resource ownership checks
- ☒ Admin protection (Founder-only routes verified)
- ☒ IDOR protection (verified across all controllers)
- ☒ Privilege escalation protection (role never client-writable)

API

- ☒ Authentication
- ☒ Authorization
- ☒ Input validation
- ☐ Schema-based output filtering (manual today, works but not schema-enforced)
- ☒ Rate limiting (now global)
- ☒ Secure errors
- ☒ CORS

Data

- ☒ HTTPS (platform-enforced)
- ☐ Encryption at rest — application-level (relies on GCP platform default)
- ☒ Sensitive data minimization (no unnecessary fields returned to lower roles)
- ☒ Secure storage (Admin-SDK-mediated only)
- ☐ Verified backup strategy (UNKNOWN from source)

Infrastructure

- ☒ HTTPS
- ☐ Verified cloud IAM configuration (UNKNOWN from source)
- ☒ Secrets management (gitignored, never in source)
- ☐ CI/CD-gated deployment

Monitoring

- ☒ Security logs (admin actions, failed logins)
 - ☒ Audit logs
 - ☒ Failed login monitoring
 - ☐ Alerting
 - ☐ Formal incident response plan
-

53. Complete Feature Security Map

Unchanged from `docs/DEEP_AUDIT_REPORT.md` §23 — every feature's frontend/API/DB/auth mapping still applies; only the perimeter (headers/rate-limiting) changed today, not per-feature logic.

54. New Developer Explanation

Start at `main/backend/server.js` — it's short and reads top to bottom: load config, fail fast if `JWT_SECRET` is missing, add security headers and rate limiting, set up CORS, mount ten route files under `/api/*`, then a catch-all error handler. Every route file in `main/backend/routes/` pairs one URL prefix with `authMiddleware` (checks who you are) and usually `roleMiddleware` (checks what your role can do), before handing off to a controller in `main/backend/controllers/`. Controllers are where the actual logic lives — read/write Firestore, check that you own the thing you're editing, and return JSON. The frontend (`main/frontend/src`) is a normal React app: `App.jsx` sets up routing and wraps everything in Context providers that fetch data through `utils/api.js`, one axios instance with your login token attached automatically. Security isn't a separate module bolted on — it's the middleware chain every request passes through before reaching your controller code, plus the ownership check your controller itself performs before any write.

55. Critical Findings

None. No CRITICAL-severity finding was identified before or after today's fixes. The highest-priority remaining item is triaging the 9 existing `npm audit` findings (§31, §47 R1) — worth doing soon, not an active incident.

56. Final Security Assessment

Project-Ticket entered this audit with solid foundations and exited it with its two biggest gaps — missing security headers and thin/broken rate limiting — closed. What remains is process maturity (CI/CD, structured logging, MFA) rather than architecture or code-level vulnerabilities. Every access-control claim in this report was verified against the actual controller code, not assumed from route names or comments; nothing was found to contradict the codebase's own stated design intent.

Final Questions — Answered Directly

1. **What are we doing RIGHT?** Server-side enforcement of every access decision that matters (identity, role, ownership); real session revocation; brute-force lockout; clean secret hygiene; no

XSS/injection sinks found anywhere in the codebase.

2. **What are we doing WRONG?** Nothing actively dangerous — the gaps are absence of process controls (CI/CD, structured logging, MFA), not broken logic.
3. **Controls already implemented?** See §45 — authentication, authorization, secrets, CORS, headers (as of today), rate limiting (as of today), file validation, audit logging.
4. **Controls partially implemented?** General logging/monitoring, dependency scanning (manual only, not automated), TLS/backup verification (platform-assumed, not independently confirmed).
5. **Controls completely missing?** MFA, CI/CD security gate, structured/queryable general logging.
6. **Biggest security risk?** The 9 untriaged `npm audit` findings — the most concrete, unresolved item in this report.
7. **Can unauthorized users access protected resources?** No path found — every protected route requires a valid, non-revoked JWT.
8. **Can one user access another's data?** No — verified across all controllers; the one intentional exception (`searchByToken`) requires guessing a high-entropy token.
9. **Can privileges be escalated?** No path found — role is never client-writable outside a Founder-gated, allow-listed admin flow.
10. **Are passwords securely protected?** Yes — hashing fully delegated to Firebase Auth; app never stores or sees a raw password beyond the login request itself.
11. **Are auth tokens securely handled?** Adequately — JWT in browser storage (standard for this architecture), revocable server-side, no XSS sink exists to steal it via this app's own code.
12. **Are APIs properly protected?** Yes, following today's fixes — auth, authorization, rate limiting, and headers all in place.
13. **Is sensitive data adequately protected?** Yes at the access-control layer; no additional application-level encryption exists, which is a reasonable choice given the access-control model, not a gap.
14. **Are secrets properly managed?** Yes — confirmed clean, gitignored, never in source.
15. **Are third-party integrations secure?** Yes — narrow, well-scoped (Firebase, SMTP), no risky integrations found.
16. **Is the database properly protected?** Yes — Admin-SDK-only access model is sound as implemented.
17. **Is the infrastructure properly secured?** Largely platform-delegated to Vercel/Firebase; specifics (IAM, backup policy) are UNKNOWN from source alone.
18. **Is the project production-ready?** Mostly ready — see §50.
19. **What MUST be fixed before production?** Nothing blocking — the app is already effectively in production use. Triage the `npm audit` findings soon.
20. **What SHOULD be fixed after (i.e., soon)?** CI/CD gate, structured logging, MFA evaluation — Phase 2/3 of §51.
21. **What security architecture should we follow going forward?** Keep the current pattern: every new endpoint gets a route-level role check plus an in-controller ownership check before any mutation, exactly as every existing controller does — that consistency is this codebase's biggest security asset.